

Paper阅读: An Empirical Evaluation of In-Memory Multi-Version Concurrency Control

目录

- **摘要**
- **并发控制协议**
 - MVTO (Timestamp Ordering)
 - MVOCC (Optimistic Concurrency Control)
 - MV2PL (Two-phase Locking)
 - Serialization Certifier
- **3.多版本存储**
 - Append-Only 追加存储 (Postgresql)
 - Time-Travel Storage 时序存储 (Sql Server)
 - Delta Storage (Oracle / Mysql)
- **4.垃圾回收**
 - Tuple Level 回收
 - Transaction-level 回收
- **5.索引管理**
 - Logical Pointers
 - Physical Pointers

1.摘要

MVCC 是当今数据库领域最流行的并发控制实现。当今主流的数据库都支持MVCC。

	Year	Protocol	Version Storage	Garbage Collection	Index Management
Oracle [4]	1984	MV2PL	Delta	Tuple-level (VAC)	Logical Pointers (TupleId)
Postgres [6]	1985	MV2PL/SSI	Append-only (O2N)	Tuple-level (VAC)	Physical Pointers
MySQL-InnoDB [2]	2001	MV2PL	Delta	Tuple-level (VAC)	Logical Pointers (PKey)
HYRISE [21]	2010	MVOCC	Append-only (N2O)	-	Physical Pointers
Hekaton [16]	2011	MVOCC	Append-only (O2N)	Tuple-level (COOP)	Physical Pointers
MemSQL [11]	2012	MVOCC	Append-only (N2O)	Tuple-level (VAC)	Physical Pointers
SAP HANA [28]	2012	MV2PL	Time-travel	Hybrid	Logical Pointers (TupleId)
NuoDB [3]	2013	MV2PL	Append-only (N2O)	Tuple-level (VAC)	Logical Pointers (PKey)
HyPer [36]	2015	MVOCC	Delta	Transaction-level	Logical Pointers (TupleId)

本篇论文围绕MVCC的主要技术点进行讨论，包括：

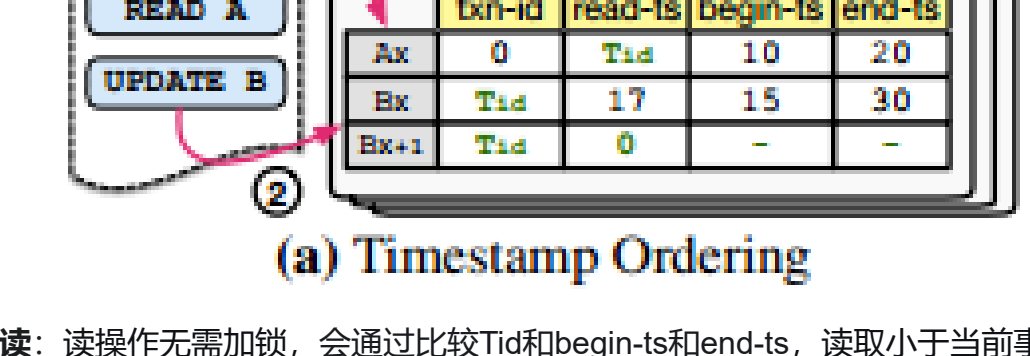
- 1. 并发控制协议
- 2. 多版本存储
- 3. 垃圾回收
- 4. 索引管理



2. 并发控制协议

MVTO （Timestamp Ordering）

1970年被提出，是最初的多版本并发控制协议。

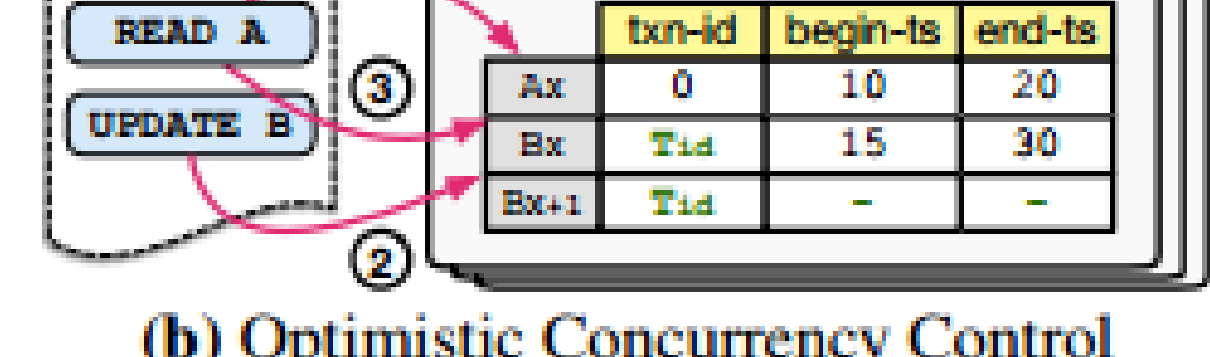


读：读操作无需加锁，会通过比较Tid和begin-ts和end-ts，读取小于当前事务时间戳的最新已提交数据版本。

写：当两个事务写同一数据项时，系统会比较它们的时间戳。如果后到的事务（时间戳更大）试图写入的数据已被先到的事务（时间戳更小）修改但尚未提交，后到的事务会被立即中止。

MVOCC （Optimistic Concurrency Control）

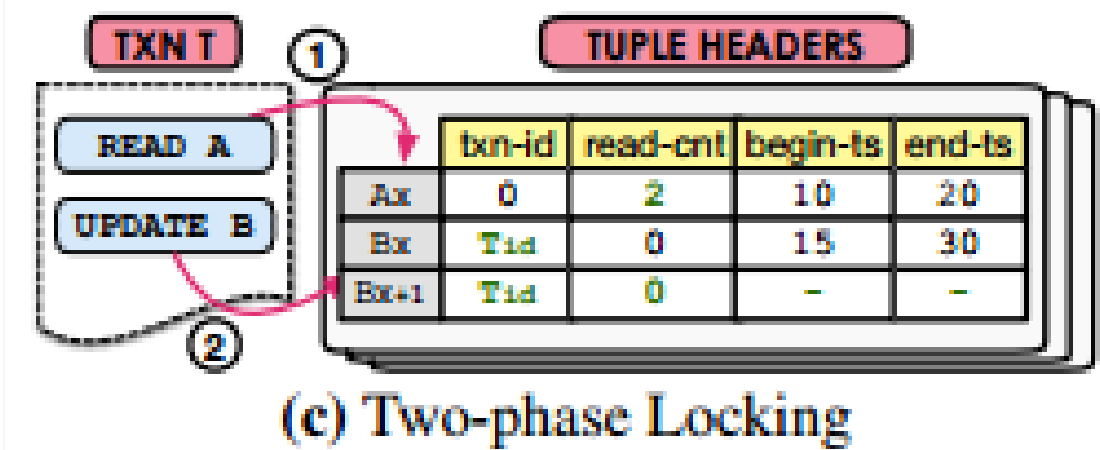
1981年提出的乐观并发控制（OCC）方案。



事务处理会分为主要三个阶段，

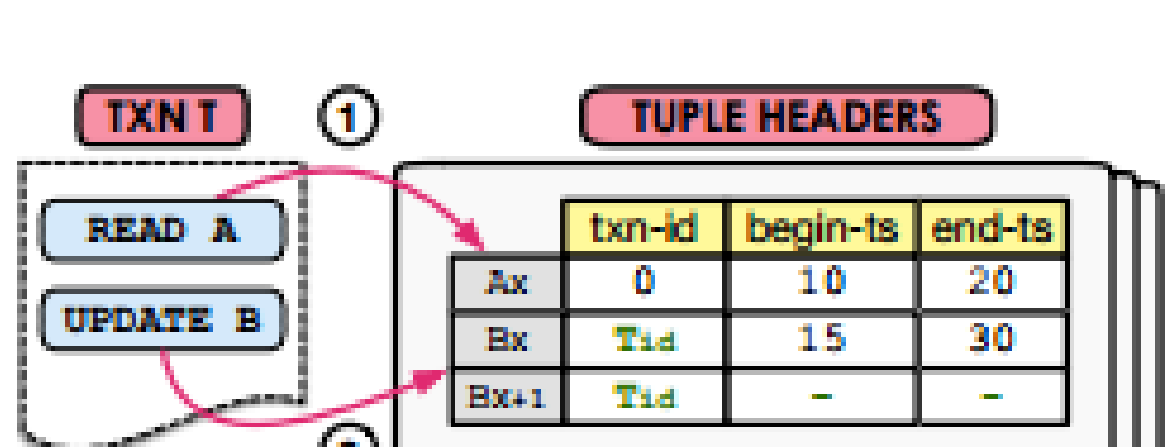
- 1. read阶段：事务读取数据库时，会看到最新的已提交数据，同时需要将需要修改的数据项复制到自己的私有工作区中。在此阶段，所有写操作都只应用在这个私有副本上，对其它事务完全不可见。
- 2. validation 阶段：验证规则通常基于事务的时间戳或版本号，确保可串行化等价性。如果验证失败，意味着发生了“读-写冲突”，该事务必须被中止并回滚。（论文中没有详细介绍规则，后面再研究一下）
- 3. commit 阶段：事务将私有工作区中的修改原子性地写入数据库的公共区域，并使之对其他事务可见，从而完成提交。

MV2PL （Two-phase Locking）



应用范围最多的，事务读写或者创建数据版本都需要获得对应的锁。

Serialization Certifier



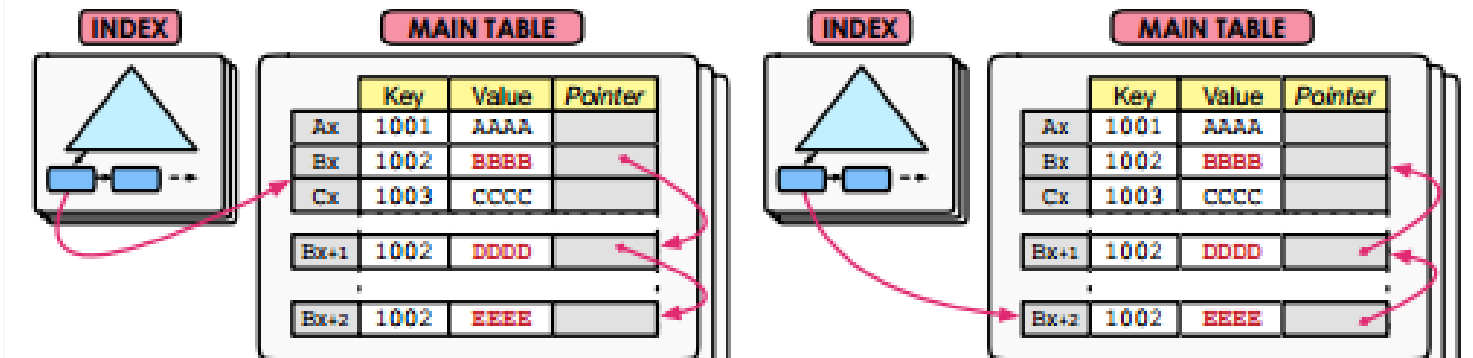
基于图的检测。PostgreSQL 是 SSI 最著名的实践者。从9.1版本开始，它提供了一个真正的 SERIALIZABLE 隔离级别，其底层实现就是SSI。SSI 的工作机制可以概括为以下三步：

- 1. 跟踪所有读写依赖关系SSI 系统会为每个事务记录两个关键集合：
 - 读集：该事务读取了哪些数据行。
 - 写集：该事务修改了哪些数据行。同时，系统会跟踪事务间的先后依赖关系，主要是：
 - 读写依赖：事务T2 读取了 事务T1 写入的数据。
 - 写读依赖：事务T2 修改了 事务T1 读取过的数据（这正是“写偏斜”的关键信号）。
- 2. 检测“危险结构”SSI 持续分析事务间的依赖图。它发现，如果在一个依赖环中，连续出现两个“写读依赖”，那么这个调度就很可能不是可串行化的。
- 3. 检测“危险结构”当检测到上述“危险结构”时，SSI会选择中止环中的一个事务（通常是最晚发起或最容易回滚的那个）。被中止的事务会回滚并可以安全重试，从而打破环，保证最终所有成功事务的调度是可串行化的。

3.多版本存储

Append-Only 追加存储 (Postgresql)

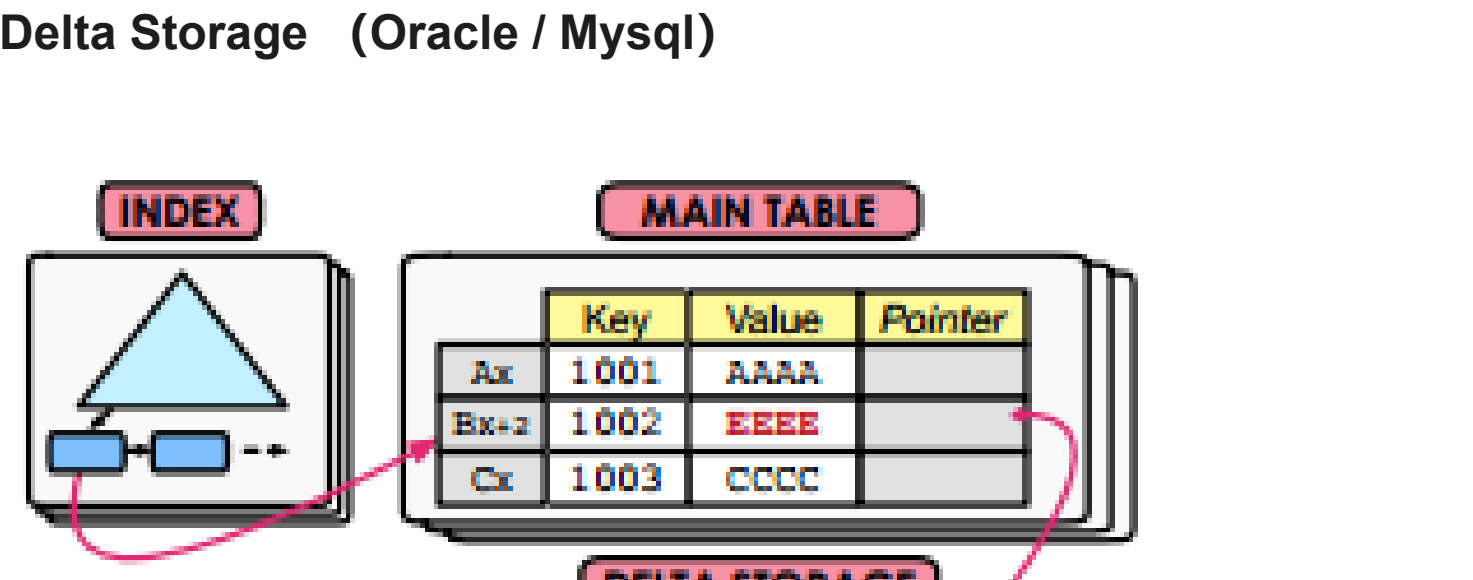
- 所有的版本存储在同一个表空间
- 更新的时候追加在版本链表上追加新节点
- Oldest-to-Newest: 链表head指向最老的
- Newest-to-Oldest: 链表head指向最新的



Time-Travel Storage 时序存储 （Sql Server）

- 每次更新的时候将之前的版本放到旧表空间
- 更新主表空间中的版本
- 优势是索引指向不需要去更新，因为索引总是指向主版本

Delta Storage （Oracle / Mysql）



- 每次更新近存储修改的部分， 将其加入链表， 主表空间存储当前版本
- 通过旧的修改部分，可以创建旧版本

4.垃圾回收

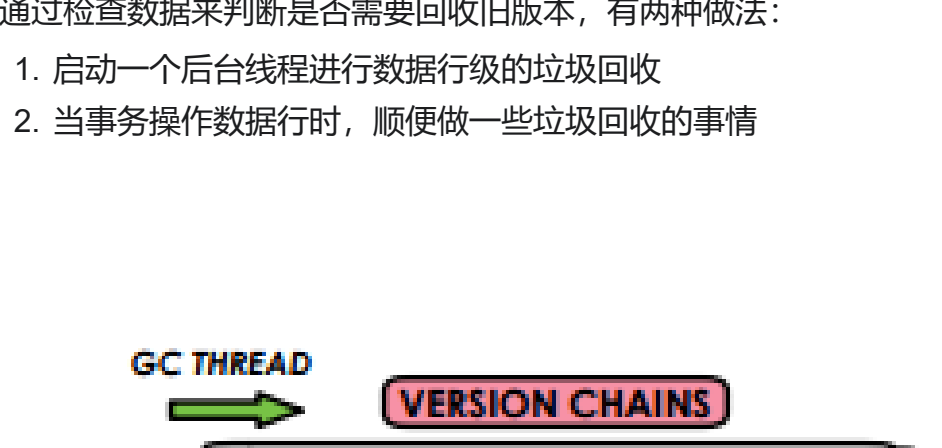
过期的旧版本：

- 1. 对应的数据上没有活跃的事务
- 2. 某版本数据的创建事务被终止

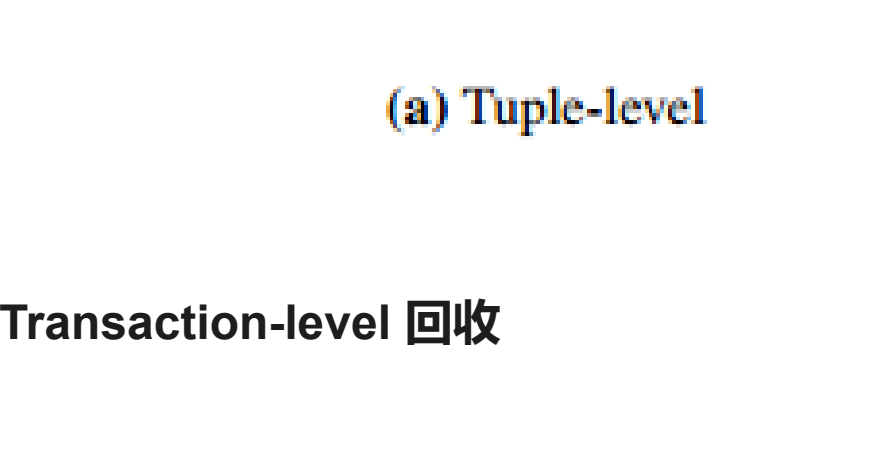
Tuple Level 回收

通过检查数据来判断是否需要回收旧版本， 有两种做法：

- 1. 启动一个后台线程进行数据行级的垃圾回收
- 2. 当事务操作数据行时，顺便做一些垃圾回收的事情



Transaction-level 回收

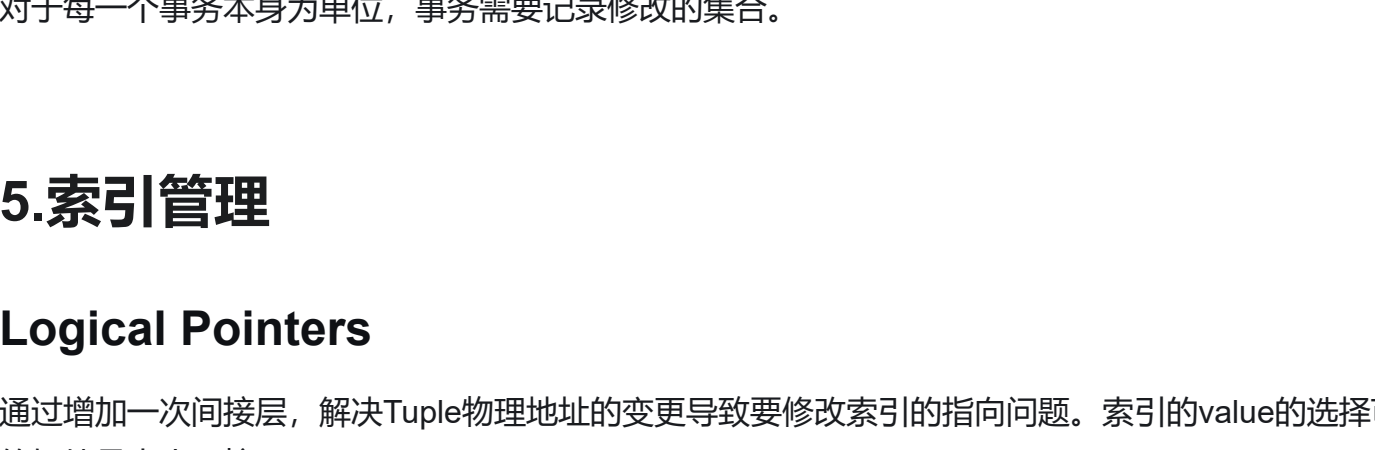


对于每一个事务本身为单位，事务需要记录修改的集合。

5.索引管理

Logical Pointers

通过增加一次间接层，解决Tuple物理地址的变更导致要修改索引的指向问题。索引的value的选择可以是主键，或者是tupleid隐式主键。隐式主键的好处是大小可控。



Physical Pointers

直接指向存储tuple的物理地址，只适用于Append-Only的Version Storage(当修改一个Tuple，会将新版本直接插入二级索引)

